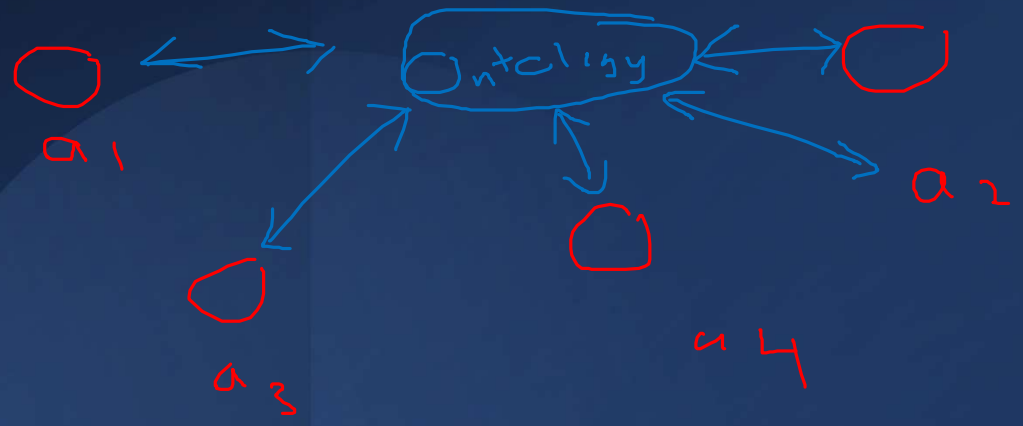


Agent Ontology

Dr. Kaneeka Vidanage

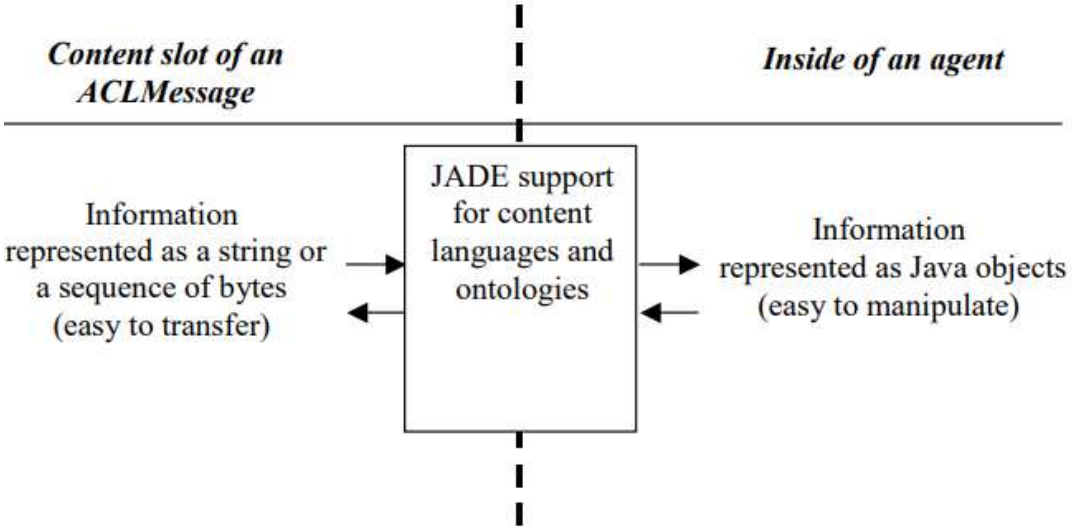
Dept of CS, FOC, KDU

Protege

What is an Ontology ? Why needed in MAS ?

- A schematic conceptualization of a particular domain, which is machine understandable.
- Ontology acts as the brain and nerve system linking agents in a Multi Agent System (MAS).
- Ontology provides the structure and meaning to the agents associated with a specific MAS.
- Ontology collaborates the functionality of different agents associated with different responsibilities to function as a one coordinated MAS.

Communication inside a MAS



- Information can conveniently be represented inside an agent as Java objects.
- ACL Templates contain msgs as string sequences or byte streams, hence it`s easy for the transmission.
- However, when it reaches to the agents, if the information inside the ACL msgs can be converted to respective Java objects, processing will be much more effective and efficient.
- JADE supports this string / byte sequences to Java object mappings and vice versa with the help of the ontology defined as well.

Information Representation.

- Inside the ACL Template.
  - "***There is a person whose name is Giovanni and who is 33 years old***" could be represented as the string **(Person :name Giovanni :age 33)** inside an ACL msg.
- Inside the Java Agent → Object based representation.

```
class Person {
      String name;
      int age;

      public String getName() {return name; }
      public void setName(String n) {name = n;
      public int getAge() {return age; }
      public void setAge(int a) {age = a; }
      ….
}
```

initialized with

```
name = "Giovanni";
age = 33;
```

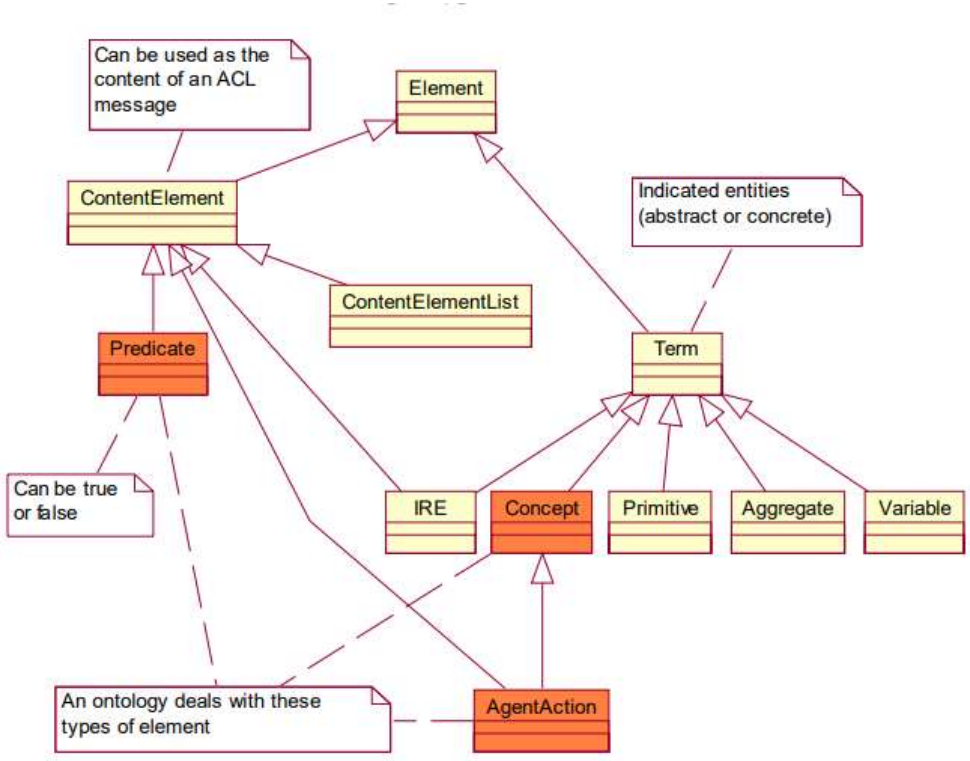
Content Manager

- Content manager object (i.e., an instance of the ContentManager class included in the jade.content package) uses Ontology and Codec interfaces to carry out ACL msgs to Java Object transformation and vice versa.
  - Ontology → Does, domain specific semantic verifications.
  - Codec → Supports mapping of ACL msgs to Java Objects and vice-versa
- Content manager utilizes a reference model architecture to methodically carry out the respective transformations as discussed above.

Content Reference Model Components.

- **Predicates** →Says something about the status of the world and can be true or false.
  - (**Works-for** (Person :name John) (Company :name TILAB)) stating that "the person John **works for** the company TILAB"
- **Entities** →Things that exists in the domain of concern and required for the modelling. Entities are further sub-divided as :-
  - **Concepts** →(**Book** :title "The Lord of the rings")
    (**Person** :name John)
    (**author** (Person :name "J.R.R. Tolkjien") → Composite concept.
  - **Agent Actions** → (**Sell** (Book :title "The Lord of the rings") (Person :name John))
  - **Identifying Referential Expressions (IRE)**→Conditional expressions
    - (all ?x (Works-for ?x (Company :name TILAB)) → All people works for company TILAB
- **Content Reference Model architecture must be followed in creating the agent ontology for the domain of concern.**

Content Reference Model



Getting ready to construct Music Ontology Example

```
package musicShopOntology;

import jade.content.onto.*;
import jade.content.schema.*;

public class MusicShopOntology extends Ontology {
  // The name identifying this ontology
  public static final String ONTOLOGY_NAME = "Music-shop-ontology";

  // VOCABULARY
  public static final String ITEM = "Item";
  public static final String ITEM_SERIAL = "serial-number";

  public static final String CD = "CD";
  public static final String CD_NAME = "name";
  public static final String CD_TRACKS = "tracks";

  public static final String TRACK = "Track";
  public static final String TRACK_TITLE = "title";
  public static final String TRACK_DURATION = "duration";

  public static final String BOOK = "Book";
  public static final String BOOK_TITLE = "title";

  public static final String OWNS = "Owns";
  public static final String OWNS_OWNER = "owner";
  public static final String OWNS_ITEM = "item";

  public static final String SELL = "Sell";
  public static final String SELL_BUYER = "buyer";
  public static final String SELL_ITEM = "item";

  // The singleton instance of this ontology
  private static Ontology theInstance = new MusicShopOntology();

  // This is the method to access the singleton music shop ontology object
  public static Ontology getInstance() {
    return theInstance;
  }
```

Singleton patterns assures same ontology object is used across the entire program and prevents creation of multiple ontology objects

Initializing the Agent Ontology construction via Private constructer adhering to Singleton pattern

```
// Private constructor
private MusicShopOntology() {
  // The music shop ontology extends the basic ontology
  super(ONTOLOGY_NAME, BasicOntology.getInstance());

  try {
    add(new ConceptSchema(ITEM), Item.class);
    add(new ConceptSchema(CD), CD.class);
    add(new ConceptSchema(TRACK), Track.class);
    add(new ConceptSchema(BOOK), Book.class);
    add(new PredicateSchema(OWNS), Owns.class);
    add(new AgentActionSchema(SELL), Sell.class);

    // Structure of the schema for the Item concept
    ConceptSchema cs = (ConceptSchema) getSchema(ITEM);
    cs.add(ITEM_SERIAL, (PrimitiveSchema) getSchema(BasicOntology.INTEGER),
           ObjectSchema.OPTIONAL); // The serial-number slot is optional and
                                   // allowed values are integers.

    // Structure of the schema for the CD concept
    cs = (ConceptSchema) getSchema(CD);
    cs.addSuperSchema((ConceptSchema) getSchema(ITEM));
    cs.add(CD_NAME, (PrimitiveSchema) getSchema(BasicOntology.STRING));
    cs.add(CD_TRACKS, (ConceptSchema) getSchema(TRACK), 1,
           ObjectSchema.UNLIMITED); // The tracks slot has cardinality > 1

    // Structure of the schema for the Track concept
    cs = (ConceptSchema) getSchema(TRACK);
    cs.add(TRACK_TITLE, (PrimitiveSchema) getSchema(BasicOntology.STRING));
    cs.add(TRACK_DURATION, (PrimitiveSchema)
           getSchema(BasicOntology.INTEGER), ObjectSchema.OPTIONAL);

    // Structure of the schema for the Book concept
    cs = (ConceptSchema) getSchema(BOOK);
    cs.addSuperSchema((ConceptSchema) getSchema(ITEM));
    cs.add(BOOK_TITLE, (PrimitiveSchema) getSchema(BasicOntology.STRING));

    // Structure of the schema for the Owns predicate
    PredicateSchema ps = (PredicateSchema) getSchema(OWNS);
    ps.add(OWNS_OWNER, (ConceptSchema) getSchema(BasicOntology.AID));
    ps.add(OWNS_ITEM, (ConceptSchema) getSchema(ITEM));

    // Structure of the schema for the Sell agent action
    AgentActionSchema as = (AgentActionSchema) getSchema(SELL);
    as.add(SELL_ITEM, (ConceptSchema) getSchema(ITEM));
    as.add(SELL_BUYER, (ConceptSchema) getSchema(BasicOntology.AID));
  }
  catch (OntologyException oe) {
    oe.printStackTrace();
  }
}
```

Primitive Schema represents basic data types as integers, strings, floats etc.

Concepts, Predicates, Actions are introduced and each of these components are connected with a respective Java bean class for information handling via getters and setters

Defining Java Bean Classes

- All concepts defined in the agent ontology will be linked with a respective Java bean class for the information handling.
- If the schema is a **ConceptSchema** the class must implement (directly or indirectly) the Concept interface
- If the schema is a **PredicateSchema** the class must implement (directly or indirectly) the Predicate interface
- If the schema is a **AgentActionSchema** the class must implement (directly or indirectly) the AgentAction interface.

Company Concept Bean Class Example

```
public class Company implements Concept {

        private String  _name;                                  //Company's name
        private Address _address;                               //Headquarter's address

        // Methods required to use this class to represent the COMPANY role
        public void setName(String name) {
                _name=name;
        }
        public String getName() {
                return _name;
        }
        public void setAddress(Address address) {
                _address=address;
        }
        public Address getAddress() {
                return _address;
        }

        // Other application specific methods
        public boolean equals(Company c){
                return (_name.equalsIgnoreCase(c.getName()));
        }
}
```

Works-For Predicate Bean Class Example

```
public class WorksFor implements Predicate {

        private Company _company;
        private Person  _person;

        //These methods are used by the JADE-framework
        public void setPerson(Person person) {
                _person=person;
        }
        public Person getPerson() {
                return _person;
        }

        public void setCompany(Company company) {
                _company=company;
        }
        public Company getCompany() {
                return _company;
        }

}
```

Selecting ACL Msg Content Language

- Mainly 02 content languages are available.
  - SL → Human Understandable
  - LEAP → Byte code which is human irrational.
- Depending on the language selected suitable CODEC has to be imported.
- The LEAPCodec class is lighter than the SLCodec class. When there are strong memory limitations the LEAP language is preferable.

Registering Ontology and Language Codec

```
public class SellerAgent extends Agent {
  private Codec codec = new SLCodec();
  private Ontology ontology = MusicShopOntology.getInstance();
  ….
  protected void setup() {
    ….
    getContentManager().registerLanguage(codec);
    getContentManager().registerOntology(ontology)
    ….
  }
  ….
}
```

Ontology Merging

- Sometimes, we can re-use readily available ontologies to expand and enrich the domain ontology we suppose to create.
- Usually, the domain ontology is the more specific ontology, we suppose to create. It can be extended by inheriting from a more generic ontology.
- Here, the concept Item, Predicate Owns and Action Sell, is directly associated with the E-Commerce ontology. Via ontology merging, let`s try to get these concepts incorporated with the Music Ontology.

Ontology Merging….1

```
package musicShopOntology;

import jade.content.onto.*;
import jade.content.schema.*;
import eCommerceOntology.*;

public class MusicShopOntology extends Ontology {
  // The name identifying this ontology
  public static final String ONTOLOGY_NAME = "Music-shop-ontology";

  // VOCABULARY
  public static final String CD = "CD";
  public static final String CD_NAME = "name";
  public static final String CD_TRACKS = "tracks";
}
```

Ontology Merging…..2

```
public static final String TRACK = "Track";
public static final String TRACK_TITLE = "title";
public static final String TRACK_DURATION = "duration";

public static final String BOOK = "Book";
public static final String BOOK_TITLE = "title";

// The singleton instance of this ontology
private static Ontology theInstance = new MusicShopOntology();

// This is the method to access the singleton music shop ontology object
public static Ontology getInstance() {
  return theInstance;
}
```

Ontology Merging…..3

```
// Private constructor
private MusicShopOntology() {
  // The music shop ontology extends the e-commerce ontology
  super(ONTOLOGY_NAME, ECommerceOntology.getInstance())

  try {
    add(new ConceptSchema(CD), CD.class);
    add(new ConceptSchema(TRACK), Track.class);
    add(new ConceptSchema(BOOK), Book.class);

    // Structure of the schema for the CD concept
    cs = (ConceptSchema) getSchema(CD);
    cs.addSuperSchema((ConceptSchema) getSchema(ECommerceOntology.ITEM));
    cs.add(CD_NAME, (PrimitiveSchema) getSchema(BasicOntology.STRING));
    cs.add(CD_TRACKS, (ConceptSchema) getSchema(TRACK), 1,
           ObjectSchema.UNLIMITED); // The tracks slot has cardinality > 1
    ….
```

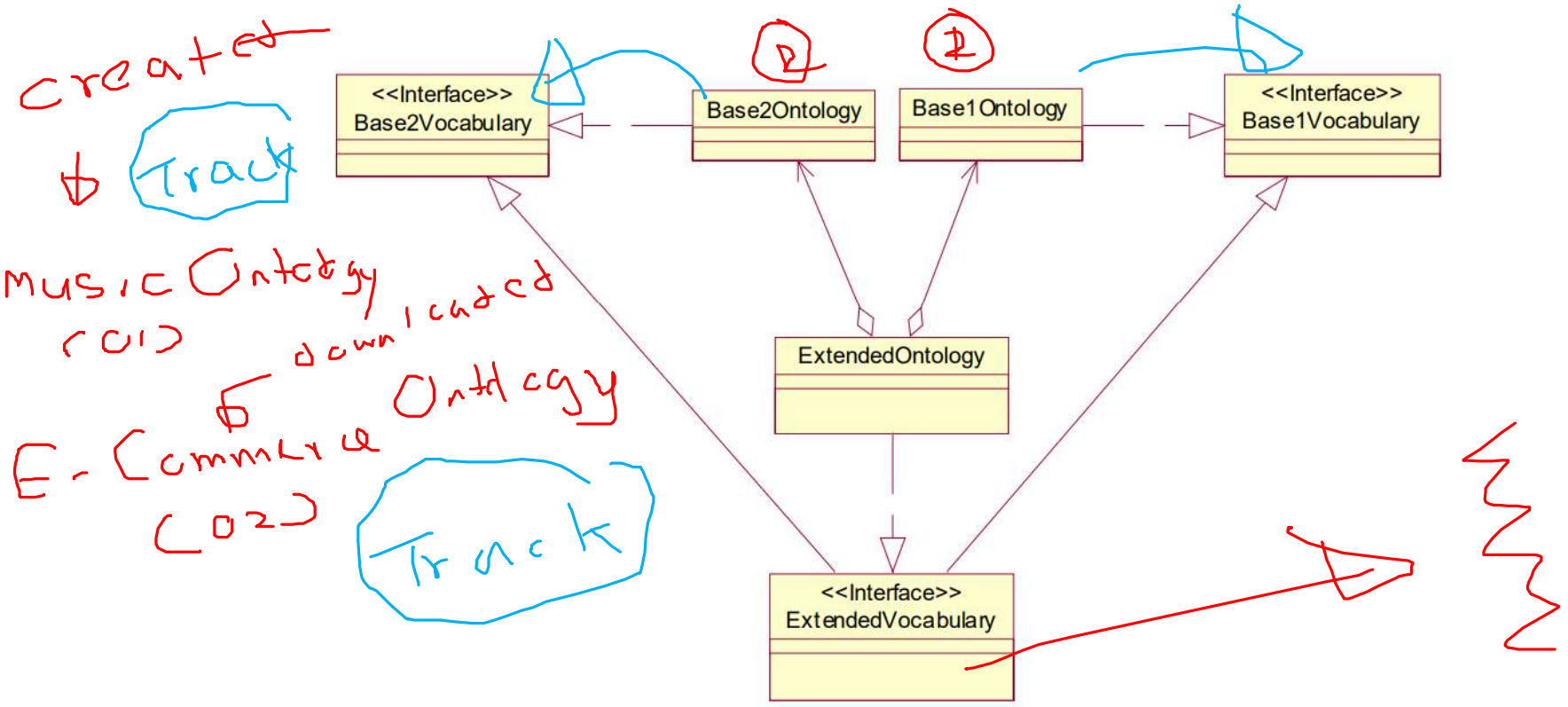
CD_Name is associated with our basic ontology, which is the music shop ontology in this case

Tracks maintain the cardinality of 1 or many

Vocabulary Interface

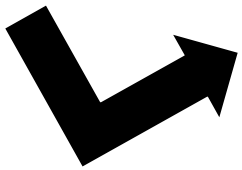
- In instances, where we merge multiple ontologies with lots of concepts, constants can be defined to properly narrate and refer them.
- Those constants can be specified in the vocabulary interface.
- Henceforth, vocabulary interface can be inherited to the extended ontologies created.
- This allows all, concepts derived from multiple places to be aggregated to one platform and allows connectivity of those to the new extended ontology as well.
- If count of the concepts are small, we have the facility of defining those inside the same class as well inform of variables, and not as an interface.

Vocabulary Interface Pattern



Expression Handling - Example

- Buyer agent asks the Seller agent if he owns “Synchronicity” CD track ?



```
// Prepare the Query-IF message
ACLMessage msg = new ACLMessage(ACLMessage.QUERY_IF);
msg.addReceiver(sellerAID); // sellerAID is the AID of the Seller agent
msg.setLanguage(codec.getName());
msg.setOntology(ontology.getName());

// Prepare the content. Optional fields are not set
CD cd = new CD();
cd.setName("Synchronicity");
List tracks = new ArrayList();
Track t = new Track();
t.setTitle("Every breath you take");
tracks.add(t);
t = new Track();
t.setTitle("King of pain");
tracks.add(t);
cd.setTracks(tracks);

Owns owns = new Owns();
owns.setOwner(sellerAID);
owns.setItem(cd);
try {
  // Let JADE convert from Java objects to string
  getContentManager().fillContent(msg, owns);
  send(msg);
}
catch (CodecException ce) {
  ce.printStackTrace();
}
catch (OntologyException oe) {
  oe.printStackTrace();
}
....
```

In the fillContent() method the Buyer agent’s content manager gets the proper Ontology and Codec objects to perform the necessary mappings to facilitate the conversion

Msg Receiving and Processing

```
….
// Receive the message
MessageTemplate mt = MessageTemplate.and(
    MessageTemplate.MatchLanguage(codec.getName()),
    MessageTemplate.MatchOntology(ontology.getName()) );
ACLMessage msg = blockingReceive(mt);

try {
  ContentElement ce = null;
  if (msg.getPerformative() == ACLMessage.QUERY_IF) {
    // Let JADE convert from String to Java objects
    ce = getContentManager().extractContent(msg);
    if (ce instanceof Owns) {
      Owns owns = (Owns) ce;
      Item it = owns.getItem();
      // Check if I have this item and answer accordingly
      ….
    }
    ….
  }
  catch (CodecException ce) {
    ce.printStackTrace();
  }
  catch (OntologyException oe) {
    oe.printStackTrace();
  }
}
….
```



Restricting the incoming messages to comply with the above configuration

Query Handling

- Working with abstract descriptors is necessary when manipulating queries.
- In particular two abstract descriptors classes are used when dealing with queries:-
  - AbsIRE - an abstract descriptor representing an Identifying Referential Expression (IRE)
  - AbsVariable - an abstract descriptor representing a Variable.
- E.g.:- Let's add the ECommerceOntology the **predicate** Costs that relates an **Item** with an integer representing the price of that item.

Query Handling

EcommerceVocabulary file

```
  …..
  public static final String COSTS = "Costs";
  public static final String COSTS_ITEM = "item";
  public static final String COSTS_PRICE = "price";
  ….
```

ECommerceOntology file

```
  ….
  // Private constructor
  private ECommerceOntology() {
    // The e-commerce ontology extends the basic ontology
    super(ONTOLOGY_NAME, BasicOntology.getInstance())

    try {
      add(new ConceptSchema(ITEM), Item.class);

add(new PredicateSchema(OWNS), Owns.class);
add(new PredicateSchema(COSTS), Costs.class);
add(new AgentActionSchema(SELL), Sell.class);
……
// Structure of the schema for the Costs predicate
PredicateSchema ps = (PredicateSchema) getSchema(COSTS);
ps.add(COSTS_ITEM, (ConceptSchema) getSchema(ITEM));
ps.add(COSTS_PRICE, (PrimitiveSchema) getSchema(BasicOntology.INTEGER));
```

Buyer agent asks the Seller agent for the price of the CD he is interested in before trying to buy it…..

```
// Prepare the Query-REF message
ACLMessage msg = new ACLMessage(ACLMessage.QUERY_REF);
msg.addReceiver(sellerAID) // sellerAID is the AID of the Seller agent
msg.setLanguage(codec.getName());
msg.setOntology(ontology.getName());

// Prepare the content.
try {
  AbsConcept absCd = ontology.fromObject(cd);
  AbsVariable x = new AbsVariable("x", BasicOntology.INTEGER);

  AbsPredicate absCosts = new AbsPredicate(MusicShopOntology.COSTS);
  absCosts.set(MusicShopOntology.COSTS_ITEM, absCd);
  absCosts.set(MusicShopOntology.COSTS_PRICE, x);

  AbsIRE absIota = new AbsIRE(SLVocabulary.IOTA);
  absIota.setVariable(x);
  absIota.setProposition(absCosts);

  // Let JADE convert from Abstract descriptor to string
  getContentManager().fillContent(msg, absIota);
  send(msg);
}
catch (CodecException ce) {
  ce.printStackTrace();
}
catch (OntologyException oe) {
  oe.printStackTrace();
}
```

Initialize the abstract concept obj from the CD object created earlier. We need abs concept to trigger parameter passing

Hence, we need the price, param is linked to the price

IOTA is needed to inject the parameter to ACL message to be passed

Defining Constraints

- Sometimes, setting of additional constraints to predicates, concepts, actions are required. This is achieved via **Facet** concept
- Facet is an instance of a class implementing the Facet interface included in the jade.content.schema.
- For an example, we can define a facet, claiming, that cost cannot be negative.

```
// Structure of the schema for the Costs predicate
PredicateSchema ps = (PredicateSchema) getSchema(COSTS);
ps.add(COSTS_ITEM, (ConceptSchema) getSchema(ITEM));
ps.add(COSTS_PRICE, (PrimitiveSchema) getSchema(BasicOntology.INTEGER));
ps.addFacet(COSTS_PRICE, new PositiveIntegerFacet());
```

Supported Bean Class to the Facet execution

```
public class PositiveIntegerFacet implements Facet {
  void validate(AbsObject abs, Ontology onto) throws OntologyException {
    try {
      AbsPrimitive p = (AbsPrimitive) abs;
      if (p.getInteger() <= 0) {
        throw new OntologyException("Integer value <= 0");
      }
    }
    catch (Exception e) {
      throw new OntologyException("Not an Integer value", e);
    }
  }
}
```